

I did not use AI for any questions besides question 4

1. **Steps**

My heuristic implementation of the step cost method is:

```
steps(state):  
    mismatches = 0  
  
    for each position i in the word:  
        if state[i] is not the same as goal[i]  
            mismatches = mismatches + 1  
  
    return mismatches
```

The heuristic counts the number of mismatches by how many letter positions are different from the goal. Since every legal action changes exactly one letter, we can infer that every mismatched position must change at least once. We also know that each step costs 1 which means the number of mismatches is a lower bound for the remaining path cost. It also cannot overestimate the true optimal remaining cost because the actual path may require additional legal actions since each state it goes through to reach the goal must exist in the dictionary. Therefore, the heuristic will underestimate, but never overestimate making it admissible.

Scrabble

My heuristic implementation of the scrabble cost method is:

```
scrabble(state):  
    estimation = 0  
  
    for each position i in the word:  
        if state[i] is not the same as goal[i]:  
            estimation = estimation + ScrabbleValue(goal[i])  
  
    return estimation
```

The heuristic adds the respective value assigned to every letter that has not been placed in the correct position from the goal. Meaning every mismatched position has to eventually be changed to the correct letter of the goal. Since each action cost is based on the letter being

replaced, the cost of replacing each of the letters to get to the goal is unavoidable. The heuristic cannot overestimate because valid paths in the dictionary might require additional letter replacements, therefore, the heuristic underestimates and can't overestimate making it admissible.

Frequency

For frequency formula is $1 + \text{rarity}(w)$, however, for my heuristic, the precise formula is:

if **state = goal** then **0**, otherwise **mismatches(state, w) + rarity(w)**

The formula for my heuristic consists of $\text{mismatches}(\text{state}, w)$ which is just the number of positions where the current word differs from the goal word. This number of mismatches would then just be added towards $\text{rarity}(w)$. The reason why this would work is because the number of mismatched positions will at least have the same number of replacements required to reach the goal. Since the original frequency formula is $1 + \text{rarity}(w)$ and rarity is nonnegative, the number of actions required by the mismatched positions would just be the step cost, making it unavoidable. Additionally, we can also see that the lower bound for frequency cost method is the minimum cost of performing one action for each mismatched letter. The heuristic also can't overestimate because it doesn't take into account additional actions required by the dictionary. Therefore, the heuristic can only underestimate the true optimal remaining cost making it admissible.

2.

Word pair	Cost measure	UCS cost	UCS node expanded	A* cost	A* nodes expanded	Reduction
Why -> Ask	Steps	5	773	5	16	97.9%
Why -> Ask	Scrabble	9	765	9	13	98.3%
Why -> Ask	Frequency	38.8	808	38.8	634	21.5%
Bear -> Duck	Steps	4	691	4	6	99.1%
Bear -> Duck	Scrabble	11	2981	11	6	99.8%
Bear -> Duck	Frequency	38.2	1086	38.2	302	72.1%

After running each problem using all three cost measures, UCS and A* both returned the same solution cost. We can see this on the chart where for the word pair Why -> Ask, the UCS cost and A* cost both have the same solution cost of 5 for the steps cost measure, 9 for the scrabble cost measure, and 38.8 for the frequency cost measure. While for the word pair Bear -> Duck, the UCS cost and A* cost both have 4 for the steps cost measure, 11 for the scrabble cost measure, and 38.2 for the frequency cost measure.

For both word pair, A* produced the largest percentage reduction using the scrabble heuristic. For Why -> Ask, the largest reduction was 98.3% and used the scrabble cost measure. While Bear -> Duck had the largest reduction of 99.8% which also used the scrabble cost measure.

For Why -> Ask, the least informative heuristic was the frequency heuristic and only had a 21.5% reduction. Similarly, for Bear -> Duck, the least informative heuristic was also the frequency heuristic and had a 72.1% reduction.

The reason why there is such a big difference between the heuristics is how they handle the actual remaining cost. For example, the steps heuristic only counts mismatched positions, so since each mismatch has to eventually be changed, this gives A* the remaining action it needs which resulted in a big reduction of 97.9% and 99.1% for the respective word pairs. For the scrabble heuristic, instead of only counting mismatches, it adds the given cost of the letters of the goal that will eventually be replaced. Allowing the heuristic to handle more on the cost of the goal which is why it produced the highest reduction for both word pairs. Lastly, for frequency heuristic, it uses the precise formula of **mismatches(state,w) + rarity(w)** which only accounts for the minimum number of actions for the unavoidable cost of eventually going to the goal state. It does not account for the rarity cost of additional actions along the path towards the goal state. Additionally, those additional actions along the path might make up a large portion of the actual cost, which is why this heuristic isn't as effective because the estimation could differ significantly with the actual remaining cost.

3.

Steps: Yes, the optimal cost from W1 to W2 is always equal to the optimal cost going from W2 to W1. Since in steps cost measure, every letter replacement cost exactly 1, therefore, going backwards would also equal the same cost. Additionally, both the initial state and goal state have to have equal length which means the optimal cost will always be the same going forward or backward.

Scrabble: No, the optimal cost isn't always equal because it depends on the given value of each letter that it has to replace going from W1 to W2 and in reverse.

Example: dog -> cog

When we replace d with c, it costs 3 to insert c into the position of d. Now when we go backward from cog -> dog, we're now inserting d into the position of c which cost 2. Both

actions are legal actions, but their cost differs based on the given value of each letter, therefore, their optimal cost won't always be equal going forward and backward.

Frequency: No, the optimal cost doesn't always have to be equal which is similar in the case of the scrabble cost measure. Much like the scrabble cost measure, the path cost depends on the rarity value of the current word to the next word.

Example: dog -> cat

When running the `python3 dcsolve.py dog cat frequency`, the cost is **23.02**.

Whereas when we run `python3 dcsolve.py cat dog frequency`, the cost is **21.96**. This clearly shows that the optimal cost isn't always equal going forward and backward.

4.

Model: ChatGPT 5.5

Precise Formula:

$$h(n) = h_0(n) + \sum_{k=0}^{h_0(n)-1} R_k$$

The heuristic proposed is admissible because from a state with a certain hamming distance, the path to the goal will eventually pass through every lower distance layer. Additionally, the heuristic only adds the minimum rarity from each unavoidable layer it passes, therefore it's always accounting for the lower bound which means it can't overestimate the actual remaining cost. It's also only adding the minimum required action to reach the goal, much like the **1 + rarity(w) formula**, making this heuristic admissible.

The proposed heuristic is a valid heuristic because every term in the formula given represents a lower bound on the path cost so the heuristic never overestimates, making this heuristic admissible and doesn't need any repair.